13
Geometry

Above the gateway to Plato's academy appeared the inscription, "Let no one who is ignorant of geometry enter here." The organizers of programming competitions feel much the same way, for you can count on seeing at least one geometric problem at every contest.

Geometry is an inherently visual discipline, one that mandates drawing pictures and studying them carefully. Part of the difficulty of geometric programming is that certain "obvious" operations you do with a pencil, such as finding the intersection of two lines, requires non-trivial programming to do correctly with a computer.

Geometry is a subject which everybody studies in high school but which often turns rusty with time. In this chapter, we will refresh your knowledge with programming problems associated with "real" geometry – lines, points, circles, and so forth. After solving a few of these problems you should feel confident enough to walk through Plato's academy once again.

There is more geometry to come. We defer problems associated with line segments and polygons to Chapter 14.

13.1 Lines

Straight *lines* are the shortest distance between any two points. Lines are of infinite length in both directions, as opposed to *line segments*, which are finite. We limit our discussion here to lines in the plane.

- *Representation* — Lines can be represented in two different ways, as either pairs of points or as equations. Every line l is completely represented by any pair of

points (x_1, y_1) and (x_2, y_2) which lie on it. Lines are also completely described by equations such as $y = mx + b$, where m is the *slope* of the line and b is the *y-intercept*, i.e., the unique point $(0, b)$ where it crosses the x-axis. The line l has slope $m = \Delta y/\Delta x = (y_1 - y_2)/(x_1 - x_2)$ and intercept $b = y_1 - mx_1$.

Vertical lines cannot be described by such equations, however, because dividing by Δx means dividing by zero. The equation $x = c$ denotes a vertical line that crosses the x-axis at the point $(c, 0)$. This special case, or *degeneracy*, requires extra attention when doing geometric programming. We use the more general formula $ax + by + c = 0$ as the foundation of our line type because it covers all possible lines in the plane:

```
typedef struct {
        double a;               /* x-coefficient */
        double b;               /* y-coefficient */
        double c;               /* constant term */
} line;
```

Multiplying these coefficients by any non-zero constant yields an alternate representation for any line. We establish a canonical representation by insisting that the y-coefficient equal 1 if it is non-zero. Otherwise, we set the x-coefficient to 1:

```
points_to_line(point p1, point p2, line *l)
{
        if (p1[X] == p2[X]) {
                l->a = 1;
                l->b = 0;
                l->c = -p1[X];
        } else {
                l->b = 1;
                l->a = -(p1[Y]-p2[Y])/(p1[X]-p2[X]);
                l->c = -(l->a * p1[X]) - (l->b * p1[Y]);
        }
}

point_and_slope_to_line(point p, double m, line *l)
{
        l->a = -m;
        l->b = 1;
        l->c = -((l->a*p[X]) + (l->b*p[Y]));
}
```

- *Intersection* — Two distinct lines have one *intersection point* unless they are *parallel*; in which case they have none. Parallel lines share the same slope but have different intercepts and by definition never cross.

```
bool parallelQ(line l1, line l2)
{
     return ( (fabs(l1.a-l2.a) <= EPSILON) &&
              (fabs(l1.b-l2.b) <= EPSILON) );
}

bool same_lineQ(line l1, line l2)
{
     return ( parallelQ(l1,l2) && (fabs(l1.c-l2.c) <= EPSILON) );
}
```

A point (x', y') lies on a line $l : y = mx + b$ if plugging x' into the formula for x yields y'. The intersection point of lines $l_1 : y = m_1 x + b_1$ and $l_2 : y_2 = m_2 x + b_2$ is the point where they are equal, namely,

$$x = \frac{b_2 - b_1}{m_1 - m_2}, \qquad y = m_1 \frac{b_2 - b_1}{m_1 - m_2} + b_1$$

```
intersection_point(line l1, line l2, point p)
{
     if (same_lineQ(l1,l2)) {
         printf("Warning: Identical lines, all points intersect.\n");
         p[X] = p[Y] = 0.0;
         return;
     }

     if (parallelQ(l1,l2) == TRUE) {
         printf("Error: Distinct parallel lines do not intersect.\n");
         return;
     }

     p[X] = (l2.b*l1.c - l1.b*l2.c) / (l2.a*l1.b - l1.a*l2.b);

     if (fabs(l1.b) > EPSILON)       /* test for vertical line */
             p[Y] = - (l1.a * (p[X]) + l1.c) / l1.b;
     else
             p[Y] = - (l2.a * (p[X]) + l2.c) / l2.b;
}
```

- *Angles* — Any two non-parallel lines intersect each other at a given angle. Lines $l_1 : a_1 x + b_1 y + c_1 = 0$ and $l_2 : a_2 x + b_2 y + c_2 = 0$, written in the general form, intersect at the angle θ given by:

$$\tan\theta = \frac{a_1 b_2 - a_2 b_1}{a_1 a_2 + b_1 b_2}$$

For lines in slope-intercept form this reduces to $\tan\theta = (m_2 - m_1)/(m_1 m_2 + 1)$.

Two lines are *perpendicular* if they cross at right angles to each other. For example, the x-axis and y-axis of a rectilinear coordinate system are perpendicular, as are the lines $y = x$ and $y = -1/x$. The line perpendicular to $l : y = mx + b$ is $y = (-1/m)x + b'$, for all values of b'.

- *Closest Point* — A very useful subproblem is identifying the point on line l which is closest to a given point p. This closest point lies on the line through p which is perpendicular to l, and hence can be found using the routines we have already developed:

```
closest_point(point p_in, line l, point p_c)
{
      line perp;              /* perpendicular to l through (x,y) */

      if (fabs(l.b) <= EPSILON) {     /* vertical line */
              p_c[X] = -(l.c);
              p_c[Y] = p_in[Y];
              return;
      }

      if (fabs(l.a) <= EPSILON) {     /* horizontal line */
              p_c[X] = p_in[X];
              p_c[Y] = -(l.c);
              return;
      }

      point_and_slope_to_line(p_in,1/l.a,&perp); /* normal case */
      intersection_point(l,perp,p_c);
}
```

- *Rays* — These are half-lines originating from some vertex v, called the *origin*. Any ray is completely described by a line equation, origin, and direction or the origin and another point on the ray.

13.2 Triangles and Trigonometry

An *angle* is the union of two rays sharing a common endpoint. *Trigonometry* is the branch of mathematics dealing with angles and their measurement.

There are two common units used to measure angles, *radians* and *degrees*. The entire range of angles spans from 0 to 2π radians or, equivalently, 0 to 360 degrees. Using radians is better computationally, because the trigonometric libraries we will see in Section 13.5 assume angles are measured in radians. However, we confess that we think more naturally in degrees. Historically, fractional parts of angles measured in degrees are given in *minutes*, or 1/60th of a degree. But working in degrees and minutes is hopeless, which is why radians (or at least decimal degrees) are the preferred measure.

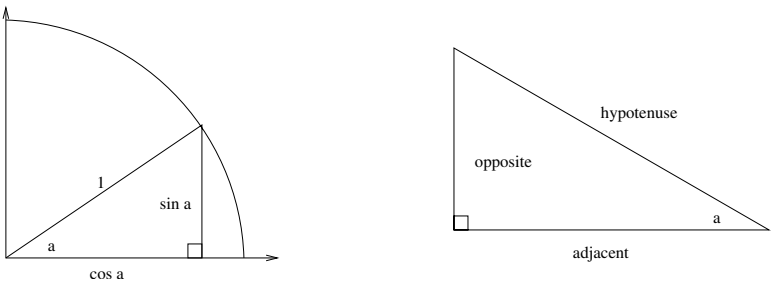


Figure 13.1. Defining sine and cosine (l). Labeling the edges of a right triangle (r).

The geometry of triangles ("three angles") is intimately related to trigonometry, so we will discuss them together in the sections below.

13.2.1 Right Triangles and the Pythagorean Theorem

A *right* angle measures 90° or $\pi/2$ radians. Right angles are formed at the intersection of two perpendicular lines, such as rectilinear coordinate axes. Such lines divide the $360^o = 2\pi$ radian space into four right angles.

Each pair of rays with a common endpoint actually defines two angles, an *internal angle* of a radians and an *external angle* of $2\pi - a$ radians. The internal angles are the ones we usually claim to be interested in. The three internal (smaller) angles of any triangle add up to $180^o = \pi$ radians, meaning that the average internal angle is $60^o = \pi/3$ radians. Triangles with three equal angles are called *equilateral*, as was discussed in Section 12.2.

A triangle is called a *right* triangle if it contains a right internal angle. Right triangles are particularly easy to work with because of the *Pythagorean theorem*, which enables us to calculate the length of the third side of any triangle given the length of the other two. Specifically, $|a|^2 + |b|^2 = |c|^2$, where a and b are the two shorter sides, and c is the longest side or *hypotenuse*.

One can go a long way in analyzing triangles with just the Pythagorean theorem. But we can go even farther using trigonometry.

13.2.2 Trigonometric Functions

The trigonometric functions *sine* and *cosine* are defined as the x- and y-coordinates of points on the unit circle centered at $(0, 0)$, as shown in Figure 13.1(l). Thus the values of sine and cosine range from -1 to 1. Further, the two functions are really the same thing, since $\cos(\theta) = \sin(\theta + \pi/2)$.

A third important trigonometric function is the *tangent*, defined as the ratio of sine over cosine. Thus $\tan(\theta) = \sin(\theta)/\cos(\theta)$, which is well-defined except when $\cos(\theta) = 0$ at $\theta = \pi/2$ and $\theta = 3\pi/2$.

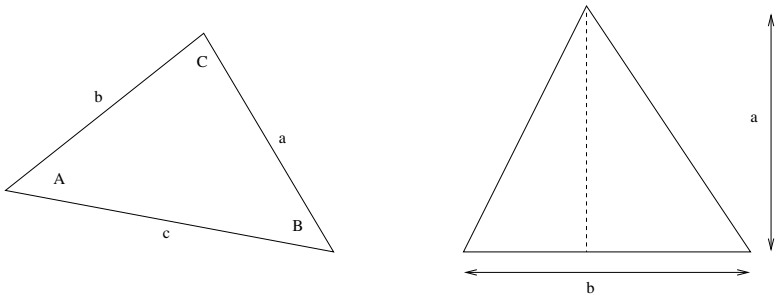


Figure 13.2. Notation for solving triangles (l) and computing their area (r).

These functions are important, because they enable us to relate the lengths of any two sides of a right triangle T with the non-right angles of T. Recall that the hypotenuse of a right triangle is the longest edge in T, that edge opposite the right angle. The other two edges in T can be labeled as *opposite* or *adjacent* edges in relation to a given angle a, as shown in Figure 13.1(r). Then

$$\cos(a) = \frac{|\text{adjacent}|}{|\text{hypotenuse}|}, \quad \sin(a) = \frac{|\text{opposite}|}{|\text{hypotenuse}|}, \quad \tan(a) = \frac{|\text{opposite}|}{|\text{adjacent}|}$$

These relations are worth remembering. As a mnemonic, we use the name of the famous Indian Chief Soh-Cah-Toa, where each syllable of his name encodes a different relation. "Cah" means *C*osine equals *A*djacent over *H*ypotenuse, for example.

Chief Soh-Cah-Toa would not be of much use without inverse functions mapping $\cos(\theta)$, $\sin(\theta)$, and $\tan(\theta)$ back to the original angles. These inverse functions are called arccos, arcsin, and arctan, respectively. With them, we can easily compute the remaining angles of any right triangle given two side lengths.

These trigonometric functions are properly computed using Taylor series expansions, but don't worry: the math library of your favorite programming language already includes them. Trigonometric functions tend to be numerically unstable, so be careful. Do not expect that θ exactly equals $\arcsin(\sin(\theta))$, particularly for large or small angles.

13.2.3 Solving Triangles

Two powerful trigonometric formulae enable us to compute important properties of triangles. The *Law of Sines* provides the relationship between sides and angles in any triangle. For angles A, B, C, and opposing edges a, b, c (shown in Figure 13.2(l)),

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

The *Law of Cosines* is a generalization of the Pythagorean theorem beyond right angles. For any triangle with angles A, B, C, and opposing edges a, b, c,

$$a^2 = b^2 + c^2 - 2bc\cos A$$

Solving triangles is the art of deriving the unknown angles and edge lengths of a triangle given a subset of such measurements. Such problems fall into two categories:

- *Given two angles and side, find the rest* — Finding the third angle is easy, since the three angles must sum to $180^o = \pi$ radians. The Law of Sines then gives us a way to find the missing edge lengths.

- *Given two sides and an angle, find the rest* — If the angle lies between the two edges, the Law of Cosines gives us the way to find the remaining edge length. The Law of Sines then enables us to mop up the unknown angles. Otherwise, we can use the Law of Sines and the angle sum property to determine all angles, and then the Law of Sines to get the remaining edge length.

The area $A(T)$ of a triangle T is given by $A(T) = (1/2)ab$, where a is the altitude and b is the base of the triangle. The base is any one of the sides while the altitude is the distance from the third vertex to this base, as shown in Figure 13.2(r). This altitude can be easily calculated via trigonometry or the Pythagorean theorem, depending on what is known about the triangle.

Another approach to computing the area of a triangle is directly from its coordinate representation. Using linear algebra and determinants, it can be shown that the *signed* area $A(T)$ of triangle $T = (a, b, c)$ is

$$2 \cdot A(T) = \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} = a_x b_y - a_y b_x + a_y c_x - a_x c_y + b_x c_y - c_x b_y$$

This formula generalizes nicely to compute $d!$ times the volume of a simplex in d dimensions.

Note that the signed areas can be negative, so we must take the absolute value to compute the actual area. This is a feature, not a bug. We will see how to use the sign of this area to build important primitives for computational geometry in Section 14.1.

```
double signed_triangle_area(point a, point b, point c)
{
        return( (a[X]*b[Y] - a[Y]*b[X] + a[Y]*c[X]
                - a[X]*c[Y] + b[X]*c[Y] - c[X]*b[Y]) / 2.0 );
}

double triangle_area(point a, point b, point c)
{
        return( fabs(signed_triangle_area(a,b,c)) );
}
```

13.3 Circles

A *circle* is defined as the set of points at a given distance (or *radius*) from its *center*, (x_c, y_c). A *disk* is circle plus its interior, i.e., the set of points a distance at most r from its center.

- *Representation* — A circle can be represented in two basic ways, either as triples of boundary points or by its center/radius. For most applications, the center/radius representation is most convenient:

```
typedef struct {
        point c;                /* center of circle */
        double r;               /* radius of circle */
} circle;
```

The equation of a circle follows directly from its center/radius representation. Since the distance between two points is defined by $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$, the equation of a circle of radius r is $r = \sqrt{(x - x_c)^2 + (y - y_c)^2}$ or, equivalently, $r^2 = (x - x_c)^2 + (y - y_c)^2$ to get rid of the root.

- *Circumference and Area* — Many important quantities associated with circles are easy to compute. Both the area A and boundary length (circumference) C of a circle depend on the magical constant $\pi = 3.1415926$. Specifically, $A = \pi r^2$ and $C = 2\pi r$. Memorizing π to many more digits is a good way to prove you are a geek. The *diameter*, or longest straight-line distance within the circle, is simply $2r$.

- *Tangents* — A line l most likely intersects the boundary of circle c at either zero or two points; the first case meaning it misses c entirely and the second case meaning it crosses the interior of c. The only remaining case is when the line l intersects the boundary of c but not its interior. Such lines are called *tangent lines*.

  The construction of a line l tangent to c through a given point O is illustrated in Figure 13.3. The point of contact between c and l lies on the line perpendicular to l through the center of c. Since the triangle with side lengths r, d, and x is a right triangle, we can compute the unknown tangent length x using the Pythagorean theorem. From x, we can compute either the tangent point or the angle a. The distance d from O to the center is computed using the distance formula.

- *Interacting Circles* — Two circles c_1 and c_2 of distinct radii r_1 and r_2 can interact in several ways. The circles will intersect if and only if the distance between their centers is at most $r_1 + r_2$. The smaller circle (say, c_1) will be completely contained within c_2 if and only if the distance between their centers plus r_1 is at most r_2. The remaining case is if c_1 and c_2 intersect each other's boundaries at two points. As shown in Figure 13.4, the points of intersection form triangles with the two centers whose edge lengths are totally determined (r_1, r_2, and the distance between the centers), so the angles and coordinates can be computed as needed.

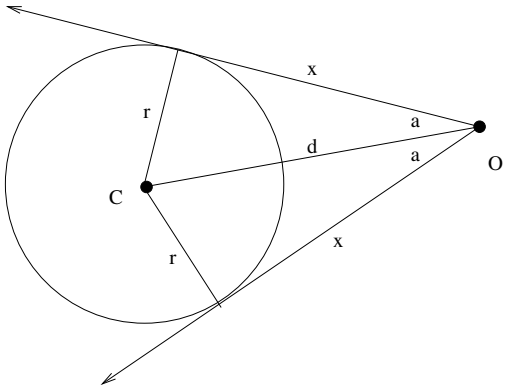


Figure 13.3. Constructing the line tangent to a circle through O.

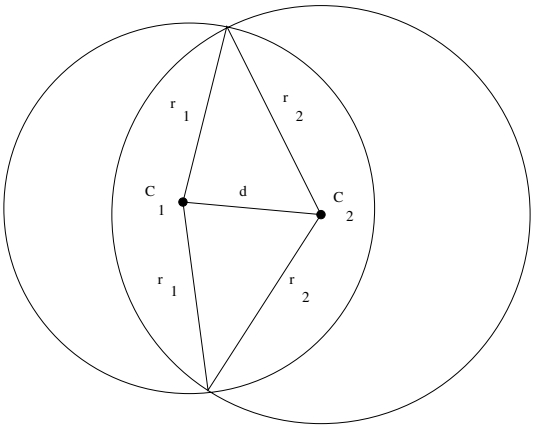


Figure 13.4. The intersection points of two circles.

13.4 Program Design Example: Faster Than a Speeding Bullet

Superman has at least two powers that normal mortals do not possess, namely, x-ray vision and the ability to fly faster than a speeding bullet. Some of his other skills are not so impressive: you or I could probably change clothes in a telephone booth if we put our minds to it.

Superman seeks to demonstrate his powers between his current position $s = (x_s, y_s)$ and a target position $t = (x_t, y_t)$. The environment is filled with circular (or cylindrical) obstacles. Superman's x-ray vision does not have unlimited range, being bounded by

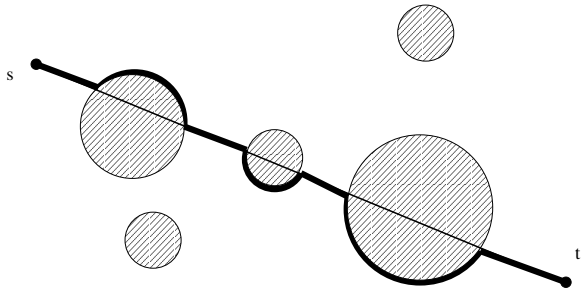


Figure 13.5. Superman's flight plan, with associated x-ray thickness.

the amount of material he has to see through. He is eager to compute the total obstacle intersection length between the two points to know whether to attempt this trick.

Failing this, the Man of Steel would like to fly between his current position and the target. He can see through objects, but not fly through them. His desired path (Figure 13.5) flies straight to the goal, until it bumps into an object. At this point, he flies along the boundary of the circle until he returns to the straight line linking position to his start and end positions. This is not the shortest obstacle-free path, but Superman is not completely stupid – he always takes the shorter of the two arcs around the circle.

You may assume that none of the circular obstacles intersect each other, and that both the start and target positions lie outside of obstacles. Circles are specified by giving the center coordinates and radius.

Solution starts below

Solving this problem requires three basic geometric operations. We need to be able to (1) test whether a given circle intersects a given line l between the start and target points, (2) compute the length of the chord intersecting l and circle, and (3) compute the arc length around the smaller piece of a circle cut by l.

The first task is relatively easy. Find the length of the shortest distance from the center of the circle to l. If this is less than the radius, they intersect; if not, they don't. To test whether this intersection occurs between s and t, it suffices to check if the point on l closest to the center of the circle lies within the box defined by s and t.

Measuring the consequences of the intersection appears more difficult. One approach would be to start by computing the coordinates of the intersection points between the line and circle. Although this could be done by setting the circle and line equations equal and solving the resulting quadratic equation, it will be a mess. *There is usually a simpler way to solve a geometric problem than explicitly finding the coordinates of points.*

Such a simpler way is revealed in Figure 13.6. The length of chord intersection is equal to $2x$ on the diagram. We know that d, the shortest length from l to the center, d, lies on a line perpendicular to l. Thus all four angles at the intersection are right

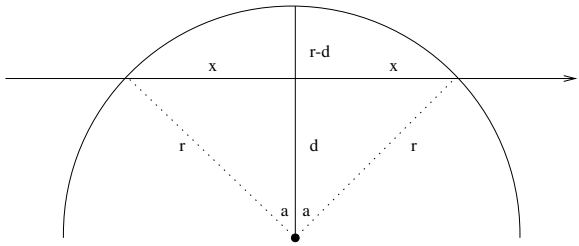


Figure 13.6. Computing chord and arc lengths for line-circle intersection.

angles, including the two angles incident on the triangles with sides r, d, and x. We can now obtain x via an application of the Pythagorean theorem.

The arc length of the shorter walk around the circle can be obtained from angle a of this triangle. The arc we are interested in is defined by the angle $2a$ (in radians), and so is $(2a)/(2\pi)$ times the total circumference of the circle, which is just $2\pi r$. The angle is easily computed from the sides of the triangle using inverse trigonometric functions.

Viewed in this way, and using the subroutines developed earlier, the solution becomes very simple:

```
point s;                        /* Superman's initial position */
point t;                        /* target position */
int ncircles;                   /* number of circles */
circle c[MAXN];                 /* circles data structure */

superman()
{
       line l;                 /* line from start to target position */
       point close;            /* closest point */
       double d;               /* distance from circle-center */
       double xray = 0.0;      /* length of intersection with circles */
       double around = 0.0;    /* length around circular arcs */
       double angle;           /* angle subtended by arc */
       double travel;          /* total travel distance */
       int i;                  /* counter */
       double asin(), sqrt();
       double distance();

       points_to_line(s,t,&l);

       for (i=1; i<=ncircles; i++) {
              closest_point(c[i].c,l,close);
              d = distance(c[i].c,close);
              if ((d>=0) && (d < c[i].r) && point_in_box(close,s,t)) {
```

```
                        xray += 2*sqrt(c[i].r*c[i].r - d*d);
                        angle = acos(d/c[i].r);
                        around += ((2*angle)/(2*PI)) * (2*PI*c[i].r);
                }
        }

        travel = distance(s,t) - xray + around;
        printf("Superman sees thru %7.3lf units, and flies %7.3lf units\n",
                xray, travel);
}
```

13.5 Trigonometric Function Libraries

The trigonometric libraries of different programming languages tend to be very similar. Make sure you know whether your library works in degrees or radians and what angle ranges are returned in the inverse trigonometric functions. Also find out which half-periods the inverse sin and cos functions assume. They cannot determine the angle over the full $360^o = 2\pi$ radian range but only a $180^o = \pi$ radian period.

Trigonometric Libraries in C/C++

The standard C/C++ math library `math.h` has all the standard trigonometric functions. Be sure to compile it with the math library included for successful operation:

```
#include <math.h>

double cos(double x);    /* compute the cosine of x radians */
double acos(double x);   /* compute the arc cosine of [-1,1] */

double sin(double x);    /* compute the sine of x radians */
double asin(double x);   /* compute the arc sine of [-1,1] */

double tan(double x)     /* compute the tangent of x radians */
double atan(double x);   /* compute the principal arctan of x */
double atan2(double y, double x); /* compute the arc tan of y/x */
```

The primary reason for two different arctan functions is correctly identifying which of the four quadrants the angle is in. This depends on the signs of both x and y.

Trigonometric Libraries in Java

The Java trigonometric functions reside in `java.lang.Math`, and assume angles are given in radians. Library functions are provided to convert between degrees and radians. All functions are static, with functionality very similar to the C library:

```
double cos(double a)     Return the trigonometric cosine of angle a.
double acos(double a)    Return the arc cosine of angle a, [0,pi].

double sin(double a)     Return the trigonometric sine of angle a.
double asin(double a)    Return the arc sine of angle a, [-pi/2,pi/2].

double tan(double a)     Return the trigonometric tangent of angle a.
double atan(double a)    Return the arc tangent of angle a, [-pi/2,pi/2]
double atan2(double a, double b)   Convert (b, a) to polar (r, theta)

double toDegrees(double angrad)   Convert a radian angle to degrees.
double toRadians(double angdeg)   Convert a degree angle to radians.
```

13.6 Problems

13.6.1 Dog and Gopher

PC/UVa IDs: 111301/10310, **Popularity:** A, **Success rate:** average **Level:** 1

A large field has a dog and a gopher. The dog wants to eat the gopher, while the gopher wants to run to safety through one of several gopher holes dug in the surface of the field.

Neither the dog nor the gopher is a math major; however, neither is entirely stupid. The gopher decides on a particular gopher hole and heads for that hole in a straight line at a fixed speed. The dog, which is very good at reading body language, anticipates which hole the gopher has chosen. The dog heads at double the speed of the gopher to the hole. If the dog reaches the hole first, the gopher gets gobbled up; otherwise, the gopher escapes.

You have been retained by the gopher to select a hole through which it can escape, if such a hole exists.

Input

The input file contains several sets of input. The first line of each set contains one integer and four floating point numbers. The integer n denotes how many holes are in the set. The four floating point numbers denote the (x, y) coordinates of the gopher followed by the (x, y) coordinates of the dog. The subsequent n lines of input each contain two floating point numbers: the (x, y) coordinates of a gopher hole. All distances are in meters to the nearest millimeter. The input is terminated by end of file and there is a blank line between two consecutive sets.

Output

Print a single line for each set of input. If the gopher can escape, the output line should read, "`The gopher can escape through the hole at (`x, y`).`" while identifying the appropriate hole to the nearest millimeter. Otherwise, the output line should read, "`The gopher cannot escape.`" If the gopher can escape through more than one hole, report the one that appears first in the input. There are at most 1,000 gopher holes in a set of input and all coordinates range between –10,000 and +10,000.

Sample Input

```
1 1.000 1.000 2.000 2.000
1.500 1.500

2 2.000 2.000 1.000 1.000
1.500 1.500
2.500 2.500
```

Sample Output

```
The gopher cannot escape.
The gopher can escape through the hole at (2.500,2.500).
```

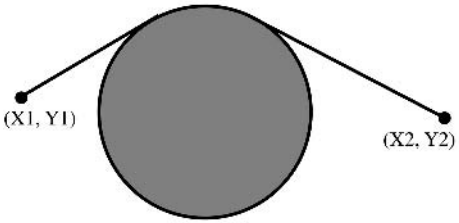
13.6.2 Rope Crisis in Ropeland!

PC/UVa IDs: 111302/10180, **Popularity:** B, **Success rate:** average **Level:** 2

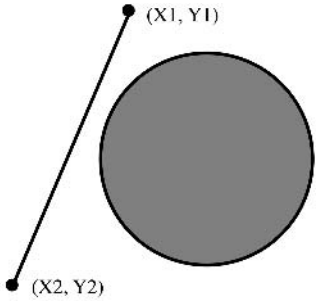
Rope-pulling (also known as tug of war) is a very popular game in Ropeland, just like cricket is in Bangladesh. Two groups of players hold different ends of a rope and pull. The group that snatches the rope from the other group is declared winner.

Due to a rope shortage, the king of the country has declared that groups will not be allowed to buy longer ropes than they require.

Rope-pulling takes place in a large room, which contains a large round pillar of a certain radius. If two groups are on the opposite side of the pillar, their pulled rope cannot be a straight line. Given the position of the two groups, find out the minimum length of rope required to start rope-pulling. You can assume that a point represents the position of each group.



Two groups with the round pillar between them.



Two groups unaffected by the pillar.

Input

The first line of the input file contains an integer N giving the number of input cases. Then follow N lines, each containing five numbers X_1, Y_1, X_2, Y_2, and R, where (X_1, Y_1) and (X_2, Y_2) are the coordinates of the two groups and $R > 0$ is the radius of the pillar.

The center of the pillar is always at the origin, and you may assume that neither team starts in the circle. All input values except for N are floating point numbers, and all have absolute value $\leq$ 10,000.

Output

For each input set, output a floating point number on a new line rounded to the third digit after the decimal point denoting the minimum length of rope required.

Sample Input	*Sample Output*
`2`	`3.571`
`1 1 -1 -1 1`	`2.000`
`1 1 -1 1 1`	

13.6.3 The Knights of the Round Table

PC/UVa IDs: 111303/10195, **Popularity:** A, **Success rate:** average **Level:** 2

King Arthur is planning to build the round table in a room which has a triangular window in the ceiling. He wants the sun to shine on his round table. In particular, he wants the table to be totally in the sunlight when the sun is directly overhead at noon.

Thus the table must be built in a particular triangular region of the room. Of course, the king wants to build the largest possible table under the circumstances.

As Merlin is out to lunch, write a program which finds the radius of the largest circular table that fits in the sunlit area.

Input

There will be an arbitrary number of test cases, each represented by three real numbers (a, b, and c), which stand for the side lengths of the triangular region. No side length will be greater than 1,000,000, and you may assume that $\max(a, b, c) \leq (a + b + c)/2$.

You must read until you reach the end of the file.

Output

For each room configuration read, you must print the following line:

`The radius of the round table is:` r

where r is the radius of the largest round table that fits in the sunlit area, rounded to three decimal digits.

Sample Input

```
12.0 12.0 8.0
```

Sample Output

```
The radius of the round table is: 2.828
```

13.6.4 Chocolate Chip Cookies

PC/UVa IDs: 111304/10136, **Popularity:** C, **Success rate:** average **Level:** 3

Making chocolate chip cookies involves mixing flour, salt, oil, baking soda, and chocolate chips to form dough, which is then rolled into a plane about 50 cm square. Circles are cut from this plane, placed on a cookie sheet, and baked in an oven for about 20 minutes. When the cookies are done, they are removed from the oven and allowed to cool before being eaten.

We are concerned here with the process of cutting the first cookie after the dough has been rolled. Each chip is visible in the planar dough, so we simply need to place the cutter so as to maximize the number of chocolate chips contained in its perimeter.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases. This line is followed by a blank line, and there is also a blank line between two consecutive test cases.

Each input case consists of a number of lines, each containing two floating point numbers indicating the (x, y) coordinates of a chip in the square surface of cookie dough. Each coordinate is between 0.0 and 50.0 (cm). Each chip may be considered a point; i.e., these are not President's Choice cookies. All of the at most 200 chocolate chips are at different positions.

Output

For each test case, the output consists of a single integer: the maximum number of chocolate chips that can be contained in a single cookie whose diameter is 5 cm. The cookie need not be fully contained in the 50-cm square dough (i.e., it may have a flat side).

The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

4.0 4.0
4.0 5.0
5.0 6.0
1.0 20.0
1.0 21.0
1.0 22.0
1.0 25.0
1.0 26.0
```

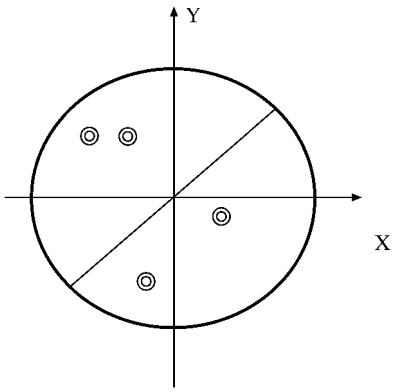
Sample Output

```
4
```

13.6.5 Birthday Cake

PC/UVa IDs: 111305/10167, **Popularity:** C, **Success rate:** average **Level:** 2

Lucy and Lily are twins. Today is their birthday, so Mother buys them a birthday cake. There are $2N$ cherries on the cake, where $1 \leq N \leq 50$. Mother wants to cut the cake into two halves with a single straight-line cut through the center so each twin gets both the same amount of cake *and* the same number of cherries. Can you help her?



The cake has a radius of 100 and its center is located at $(0, 0)$. The coordinates of each cherry are given by two integers (x, y). You must give the line in the form $Ax + By = 0$, where both A and B are integers in $[-500, 500]$. Cherries are not allowed to lie on the cutline. There is at least one solution for each data set.

Input

The input file contains several test cases. The first line of each case contains the integer N. This is followed by $2N$ lines, where each line contains the (x, y) location of a cherry with one space between them. The input is ends with $N = 0$.

Output

For each test case, print a line containing A and B with a space between them. If there are many solutions, any one will suffice.

Sample Input

```
2
-20 20
-30 20
-10 -50
10 -5
0
```

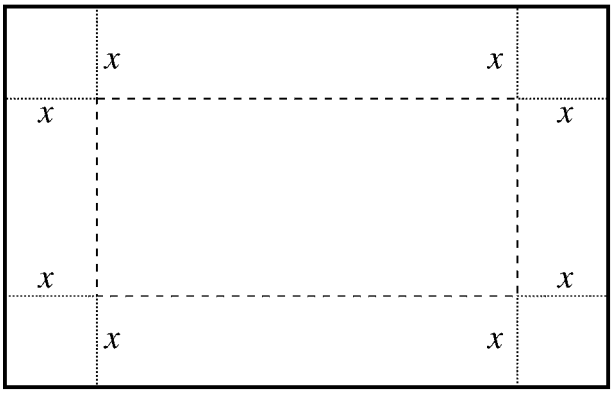
Sample Output

```
0 1
```

13.6.6 The Largest/Smallest Box ...

PC/UVa IDs: 111306/10215, **Popularity:** A, **Success rate:** average **Level:** 2

The following figure shows a rectangular card of width W, length L, and thickness 0. Four $x \times x$ squares are cut from the four corners of the card shown by the dotted lines. The card is then folded along the dashed lines to make a box without a cover.



Given the width and height of the box, find the values of x for which the box has maximum and minimum volume.

Input

The input file contains several lines of input. Each line contains two positive floating point numbers L $(0 < L < 10,000)$ and W $(0 < W < 10,000)$, which indicate the length and width of the card, respectively.

Output

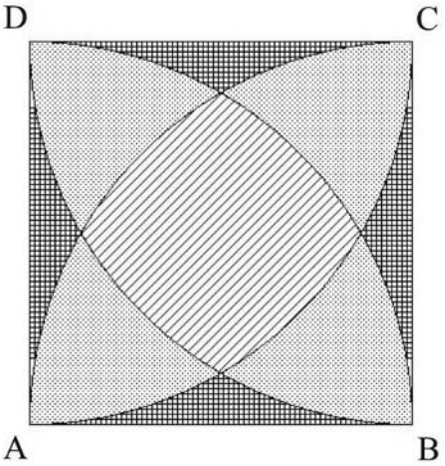
For each line of input, give one line of output containing two or more floating point numbers separated by a single space. Each floating point number should contain three digits after the decimal point. The first number indicates the value which maximizes the volume of the box, while the subsequent values (sorted in ascending order) indicate the cut values which minimize the volume of the box.

Sample Input	*Sample Output*
`1 1`	`0.167 0.000 0.500`
`2 2`	`0.333 0.000 1.000`
`3 3`	`0.500 0.000 1.500`

13.6.7 Is This Integration?

PC/UVa IDs: 111307/10209, **Popularity:** A, **Success rate:** high **Level:** 3

The image below shows a square $ABCD$, where $AB = BC = CD = DA = a$. Four arcs are drawn taking the four vertexes A, B, C, D as centers and a as the radius. The arc that is drawn taking A as center starts at neighboring vertex B and ends at neighboring vertex D. All other arcs are drawn in a similar fashion. Regions of three different shapes are created in this fashion. You must determine the total area of these different shaped regions.



Input

Each line of the input file contains a floating-point number a indicating the side length of the square, where $0 \leq a \leq 10,000.0$. Input is terminated by end of file.

Output

For each test case, output on a single line the area of the different region types in the image above. Each floating point number should be printed with three digits after the decimal point. The first number of each case will denote the area of the striped region, the second number will denote the total area of the dotted regions, and the third number will denote the rest of the area.

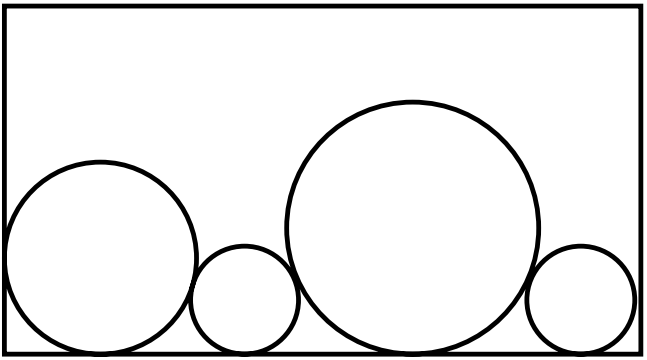
Sample Input	*Sample Output*
`0.1`	`0.003 0.005 0.002`
`0.2`	`0.013 0.020 0.007`
`0.3`	`0.028 0.046 0.016`

13.6.8 How Big Is It?

PC/UVa IDs: 111308/10012, **Popularity:** B, **Success rate:** low **Level:** 3

Ian is going to California and has to pack his things, including his collection of circles. Given a set of circles, your program must find the smallest rectangular box they fit in.

All circles must touch the bottom of the box. The figure below shows an acceptable packing for a set of circles, although it may not be the optimal packing for these particular circles. In an ideal packing, each circle should touch at least one other circle, but you probably figured that out.



Input

The first line of input contains a single positive decimal integer n, $n \leq 50$. This indicates the number of test cases to follow. The subsequent n lines each contain a series of numbers separated by spaces. The first number on each of these lines is a positive integer m, $m \leq 8$, which indicates how many other numbers appear on that line. The next m numbers on the line are the radii of the circles which must be packed in a single box. These numbers need not be integers.

Output

For each test case, your program must output the size of the smallest rectangle which can pack the circles. Each case should be output on a separate line by itself, with three places after the decimal point. Do not output leading zeroes unless the number is less than 1, e.g., `0.543`.

Sample Input	*Sample Output*

`3`
`3 2.0 1.0 2.0`
`4 2.0 2.0 2.0 2.0`
`3 2.0 1.0 4.0`
	`9.657`
	`16.000`
	`12.657`

13.7 Hints

13.1 Is the closest hole *really* the safest spot for the gopher?

13.2 Does it help to compute tangent lines to the pillar?

13.3 How many sides of the triangle must the table touch?

13.4 Can we always move the cutter circle to put some chips on its boundary? If so how many? Does this plus the radius define all "interesting" cutter placements?

13.5 There is always a solution to this problem if we remove the constraint that the cutline have integer coordinates – can you prove it? Is there a more efficient solution than trying all possible A, B pairs?

13.6 What are the values of x for which the box has zero volume? Is calculus helpful in maximizing the volume?

13.7 Can we use inclusion-exclusion to give us the area of complicated regions from easy-to-compute parts?

13.8 Is it better to order the circles from largest to smallest, or to interleave them? Does the order ever *not* matter? Will backtracking work for this problem?